Manual técnico

Robot de laberinto · Microcontroladores 2

Manuel Harari López
Víctor Miguel Vázquez
Carlos Emiliano Escamilla Garibay

Cómo leerlo. Cada sección empieza con un recuadro **En simple**, que se entiende sin ser ingeniero. Lo que sigue es el detalle técnico. Al final hay un glosario.

1. Qué hace el robot

En simple: el robot entra a un laberinto que no conoce y avanza celda por celda, mirando con tres sensores láser dónde hay pared. Busca el cuadro **amarillo** del piso, que es la meta, y se detiene al verlo. En el camino cuenta los cuadros de colores y dibuja un mapa. En el siguiente intento usa ese mapa para ir directo a la meta por el camino más corto que conoce.

- **Laberinto:** celdas de 180 mm. La meta es el cuadro amarillo; no se supone un tamaño de laberinto fijo.
- **Robot:** diferencial con dos ruedas motrices y un patín, huella ≤ 120 mm.
- **Estrategia en dos fases:**
  1. **Exploración** guiada por los sensores: de frente si se puede, si no izquierda, si no derecha, si no media vuelta. El mapa se va llenando con votos.
  2. **Ruta corta** en la siguiente corrida: si el mapa ya tiene la meta, va por el camino más corto conocido (BFS). Si algo no cuadra, sigue explorando desde donde quedó.

2. Dos cerebros

En simple: el Arduino UNO Q trae dos computadoras en una placa.

- Una **pequeña y muy puntual**, que mueve los motores y lee los sensores cien veces por segundo.
- Una **grande con Linux**, que decide el camino, guarda el mapa y mira con la cámara.

Se pasan mensajes entre ellas. El STM32 ejecuta cada movimiento por su cuenta y frena solo (a 130 mm de una pared, o al ver amarillo), así que el robot nunca queda fuera de control si Linux tarda.

Dominio	Procesador	Sistema	Tareas
Tiempo real	STM32U585 (Cortex-M33)	Zephyr (sketch Arduino)	Motores, 3 ToF, clasificación de color, MPU-6050 opcional, pantalla y botón
Decisión	Qualcomm QRB2210 (Cortex-A53)	Debian Linux	Máquina de estados, mapa, ruta corta, persistencia, cámara (OpenCV), página web

Enlace: el **Bridge** (RPC MessagePack, enrutado por `arduino-router`). Linux invoca procedimientos que expone el STM32:

Procedimiento	Qué hace
`mover(mm, lento)`	Avanza hasta *mm* (una celda = 180 mm). Si `lento` es 1 usa `VEL_LENTO`. No bloquea; devuelve 0 si ya hay un movimiento en curso
`girar(k)`	Gira en sitio *k* × 90°; el signo indica el sentido (2 = media vuelta)
`ocupado()`	1 mientras hay un movimiento activo
`odo(i)`	Lecturas: 0 = mm recorridos, 1 = mm que faltan, 2 = máscara de paredes y movimiento, 3 = saltos (1 izquierda, 2 frente, 4 derecha), 5 / 6 / 7 = distancia frente / izquierda / derecha (mm), 8 = desvío de rumbo en grados (con MPU)
`color()`	Último color confirmado (3 lecturas seguidas), una sola vez
`alto()`	Detiene el movimiento (Python lo pide al confirmar la meta)
`arranque()`	Evento del botón: 1 = corto, 2 = largo
`set_cuenta`, `set_estado`, `set_fase`	Actualizan la pantalla (conteo, estado, fase de la máquina)
`raw(i)`	Lectura cruda del sensor de color (*i* = R, G, B, C) para el panel web
`set_ref(i, r, g, b, c)`	Carga el color de referencia *i* (r, g, b en milésimas)
`set_tof(i, a, b)`	Carga la calibración del ToF *i*: real = *a* × crudo + *b* (*a* en milésimas, *b* en décimas de mm). `odo(9..11)` da la lectura sin calibrar

2.1 Página web: calibración y diagnóstico

Linux sirve una página en el puerto 7000 (brick `web_ui`), que se abre desde el celular por WiFi. Es la herramienta central de puesta a punto: toda la calibración se hace ahí, sin reprogramar ni copiar valores a mano, y queda guardada en `/app/datos/` para cargarse sola en cada arranque.

Función	Cómo funciona	Archivo
Colores	Asistente de 5 pasos; cada color promedia 5 lecturas del TCS34725 y se manda al STM32 (`set_ref`)	`colores.json`
Distancia	Por sensor, puntos (mm reales con vernier, lectura cruda promedio de 10); ajuste por mínimos cuadrados `real = a × crudo + b` (con 1 punto, solo desfase) que se manda al STM32 (`set_tof`)	`tof.json`
Simulador	Dibuja lo que mide cada ToF y lo que haría el robot (`que_haria()`) sin moverlo	
Cámara	Imagen 160 × 120 en gris con la zona de paso en rojo (blanco) o verde (negro)	
Corrida	Fase, celda, mapa y meta; durante la corrida usa la última lectura del robot y no llama al Bridge	

Se aplica en el STM32, antes de cualquier decisión, para que el control de 10 ms, los saltos y el alto a 130 mm usen milímetros reales.

Separar los dominios aísla la latencia no determinista de Linux (planificador, E/S, visión) del lazo de control.

> **Respaldo:** si el UNO Q falla, el mismo chasis acepta un UNO R3 (ATmega328P) con firmware *bare-metal* propio (carpeta `atmega328p/`) que hace todo en un solo chip, sin Arduino core. Este manual describe la versión del UNO Q.
> **Dos versiones del sketch del UNO Q:** `uno_q/sketch/` incluye el MPU-6050 (si no lo detecta al arrancar, gira y avanza por tiempo) y `uno_q/sketch_sin_mpu/` es la misma lógica sin el código del giroscopio. El Python es el mismo para las dos.

3. Sensores de distancia

En simple: tres sensores láser (izquierda, frente y derecha) miden en milímetros qué tan lejos está la pared, unas 50 veces por segundo. Una lectura rara nunca decide sola: el robot toma varias y se queda con la menor.

- **Sensores:** 3 × VL53L0X, tiempo de vuelo a 940 nm.
- **Direcciones I²C:** los tres traen la misma (0x29), que además comparte el sensor de color. Al arrancar se mantienen apagados con XSHUT (A0–A2). Se encienden uno por uno y cada uno se reasigna a 0x30–0x32 antes de `init()`.
- **Lectura:** modo continuo con *timing budget* de 20 ms (≈ 50 Hz por sensor), no bloqueante.
- **Umbrales:** lateral con menos de 120 mm (`PARED`) = pared. Frontal con menos de 130 mm (`PARED_F`) = pared.
- **Lecturas mínimas:** Python toma 3 lecturas de cada ToF y usa la menor (5 tras un cambio brusco). Una lectura mala no deja pasar al robot.

4. Sensor de color

En simple: debajo del robot hay un sensor con su propia lucecita, tapado con una capucha para que no le afecte la luz del lugar. Mide cuánto rojo, verde y azul refleja el piso y lo compara con los colores que le enseñamos en la calibración. Para no contar dos veces el mismo cuadro, exige ver el mismo color tres veces seguidas y no vuelve a contar hasta regresar al piso normal. El amarillo es la meta: el robot se detiene en cuanto lo confirma.

Adquisición

- TCS34725 con integración de 24 ms, ganancia 4× y LED propio, dentro de una capucha impresa.
- La capucha fija la distancia al piso y bloquea la luz ambiente.

Clasificación por vecino más cercano. Primero se normaliza la cromaticidad, que da invariancia de primer orden a la intensidad de luz:

> $\hat{r} = R / (R + G + B)$, igual para $\hat{g}$ y $\hat{b}$

La distancia a cada color de referencia i suma un término logarítmico de brillo. Ese término separa colores de tono parecido y distinta reflectancia, como blanco y amarillo:

> $d_i = (\hat{r} - r_i)^2 + (\hat{g} - g_i)^2 + (\hat{b} - b_i)^2 + 0.01 \cdot \ln^2((C + 1) / (C_i + 1))$

El resultado es “piso” si la menor distancia es mayor que 0.02 o si el brillo es menor que 1.3 veces el del piso negro (`BRILLO_MIN` , quita falsos positivos).

Calibración: desde un panel web que sirve la propia placa (puerto 7000). Cada referencia es el promedio de 5 lecturas tomadas sobre la pista. Se guarda en `/app/datos/colores.json` ; si no existe, se usan los valores de fábrica. Linux la carga al STM32 con `set_ref` en cada arranque, sin recompilar el sketch.

Antirrebote: se exigen 3 muestras iguales seguidas (3 × 30 ms). El evento solo se dispara en la transición piso → color.

Meta: al confirmar amarillo, el **STM32 detiene el movimiento por sí mismo**, sin esperar a Python. Python entonces guarda la celda como meta.

¿En qué celda estaba? Python calcula $k = \lfloor (mm + 36 + 90) / 180 \rfloor$, con mm los milímetros recorridos en el avance y 36 mm la distancia del eje al sensor (`Y_COLOR`). Cada celda cuenta su color una sola vez.

5. Cámara (opcional)

> **En simple:** la cámara va arriba de un poste y mira el piso por donde va a pasar. Es **monocromática** (ve en gris, no en colores), así que hace una sola cosa: detecta si el piso adelante es **blanco**, que es una pared (el piso es negro). Si ve blanco, el robot avanza más despacio, y el sensor láser frontal confirma si de verdad hay pared. La cámara nunca manda sobre los láser. Sin cámara, el robot funciona igual.

- **Captura:** innomaker U20CAM-9281M (OV9281, 1 MP monocromática, obturador global, hasta 720p a 120 fps, lente M12 gran angular), UVC a 320 × 240, OpenCV con V4L2. El obturador global evita la deformación por movimiento (*rolling shutter*) de las cámaras comunes.
- **Blanco (pared):** fracción de píxeles con gris ≥ `BLANCO_MIN` (170 de 255) en `ZONA_PASO` , el piso por donde va a pasar. Si pasa de `BLANCO_MAX` (15 %), Python pide `mover(mm, 1)` : avance lento a `VEL_LENTO` . **Doble check:** Python también lee el frontal (`odo(5)`) y lo reporta; el sketch para a 130 mm si la pared es real. Si la cámara ve blanco y el frontal no ve nada, igual va lento.
- **Sin amarillo:** el sensor OV9281 no tiene filtro de color (matriz de Bayer), así que no puede distinguir el amarillo por su tono. Antes se buscaba con HSV para desempatar islas; se quitó al cambiar a esta cámara. La meta siempre la confirmó el TCS34725, así que no se pierde capacidad. A cambio, el sensor monocromático recibe toda la luz en cada píxel (más sensibilidad y menos ruido), y el contraste blanco/negro, que es lo único que se usa, sale más limpio.
- **Montaje:** módulo (medida por confirmar; el CAD usa 32 × 32 mm) sobre un pedestal impreso de 20 × 20 × 75 mm, inclinado hacia abajo para ver el piso adelante del eje. Se conecta por un hub USB-C con entrada de alimentación: con la Q alimentada por VIN, su

USB-C no da 5 V al hub, así que el hub se alimenta por su puerto PD.

- **Ajuste:** el panel web muestra la imagen a 160 × 120 (JPEG calidad 40) con el camino en rojo (blanco) o verde (negro). Un candado comparte la cámara entre el robot y el panel. Al arrancar la busca una vez; un hilo revisa cada 2 s si sigue conectada y la vuelve a buscar si se desconecta (OLED `SIN CAM`).

6. Cómo se mueve

> **En simple:** el robot no tiene sensores en las ruedas, así que mide cuánto avanza con el láser de enfrente: anota a qué distancia está la pared al arrancar y para cuando esa distancia bajó 18 cm (una celda). Si no hay pared a la vista, avanza por tiempo. Para los giros usa un giroscopio si lo tiene, y si no, tiempo. Frena siempre a 13 cm de una pared.

Actuación

- Dos micromotorreductores DC (≈ 12 × 11 mm de sección), ruedas de 45 mm, puente H L298N.
- PWM en ENA/ENB y sentido en IN1–IN4. El sentido de cada motor se invierte por software (`INV_IZQ`, `INV_DER`).
- Freno activo: ambas entradas en alto con EN = 255 durante 60 ms.

Avance (lazo a 100 Hz, $T = 10$ ms):

1. **Por distancia.** Al iniciar toma el frontal; si lee menos de 1 m (`RANGO`) avanza hasta que haya bajado 180 mm (`CELDA`). Si no ve nada a ese alcance avanza por tiempo (`MS_CELDA`) y, si a medio camino aparece una pared, sigue por distancia.
2. **Alto y retroceso.** Se detiene al leer 130 mm al frente (`D_FRENTE`). Si queda por debajo de 120 mm (`D_FRENTE − D_CERCA`) se hace para atrás hasta 130 mm. Si hay pared y la distancia ya no baja en 300 ms, también para. Desde 230 mm frena de forma proporcional (`FRENA_K`).
3. **Perfil de velocidad.** Rampa de +6 PWM por ciclo hasta `VEL_EXPLORA` (o `VEL_LENTO`), y desaceleración lineal en el último cuarto de celda, con piso en `VEL_MIN`.
4. **Cambios bruscos.** Si una lectura de un ToF cambia `SALTO` = 120 mm o más y la siguiente lo confirma, se reconoce un cambio brusco (se abrió un lateral o saltó el frontal).
   - Si se abre un lateral, el robot avanza `TRAS_SALTO` = 124 mm más y para en el centro de esa celda, para escanearla (fase `ESCANEA`).
   - Si salta el frontal, recalcula lo que falta con la nueva lectura.
   - Python lo lee con `odo(3)`.
5. **Centrado solo bajo 90 mm.** Con un lateral a menos de `D_LADO_MIN` = 90 mm corrige: $u = K_c \cdot e + K_d \cdot \dot{e}$, saturado en ±40 PWM y aplicado en diferencial ($PWM_izq = v - u$, $PWM_der = v + u$). Con dos paredes cercanas $e = d_izq - d_der$; con una, contra los 90 mm. Con ambos lados lejos va derecho.

Giros

- **Con MPU-6050:** el sketch lo busca en la dirección 0x68 al arrancar (OLED muestra `G` si lo encontró) y mide el sesgo del giroscopio con el robot quieto, por lo que se enciende sin moverlo.
  - **Rumbo absoluto:** cada giro va al ángulo deseado acumulado, así corrige lo que se pasó o faltó en el giro anterior. Se corta `ADELANTO` = 6° antes por la inercia.
  - **Aprende el signo:** el sentido de montaje del sensor se determina en el primer giro. Antes de ese giro no sostiene rumbo.
  - **Rumbo en recta:** durante el avance sostiene el rumbo con ganancia `KG`, además del centrado con paredes.
- **Sin MPU-6050:** gira por tiempo (`MS_90`) a PWM constante `VEL_GIRO`. El mismo sketch sirve para los dos casos.

> **Limitación reconocida:** sin encoders, la distancia depende del frontal. En tramos largos sin pared a la vista, y en giros sin MPU-6050, el resultado depende del voltaje de la batería y de la fricción.
>
> - **Cómo lo mitigamos:** el frontal mide el avance por distancia, el MPU-6050 corrige el rumbo, y las ruedas patinan poco por la rampa de arranque.
> - **Mejora natural:** encoders en cuadratura para medir también los tramos sin pared.

7. Cómo decide y qué mapa guarda

> **En simple:** el robot decide siempre quieto, en cuatro pasos que se repiten: **mira** (con los láser), **decide** (a dónde ir), **gira** y **avanza** una celda. Prefiere seguir de frente; si no puede, gira a la izquierda; si no, a la derecha; y si está encerrado, da media vuelta. Si pasa tres veces por el mismo lugar (da vueltas a una isla), prueba la salida que menos ha usado.

Máquina de estados (Python; la fase se ve en la OLED y en la web):

`SENSA` → `DECIDE` → `GIRA` → `AVANZA` → `SENSA`. Tras un cambio brusco, `ESCANEA` sustituye a `SENSA`.

Fase	Qué hace
`SENSA`	Toma 3 lecturas de cada ToF (la menor) y vota las paredes de la celda
`ESCANEA`	Igual, con 5 lecturas, tras un salto
`DECIDE`	Elige la salida (ver abajo)
`GIRA`	Pide `girar(k)` y espera
`AVANZA`	Pide `mover(180, lento)` y espera; lee mm recorridos y saltos

Prioridad de salidas (todas con lecturas de ToF):

1. **Frente** si el frontal lee ≥ 220 mm (`FRENTE_LIBRE`): está a la mitad entre pared en su celda (~130) y en la siguiente (~310).
2. **Izquierda** si el lateral lee ≥ 120 mm.
3. **Derecha**, con el mismo criterio.
4. **Media vuelta** si no hay ninguna.

Reglas que la modifican

- **Tras un salto:** de las salidas que llevan a una celda que aún no está en el mapa, toma la de lectura más lejana.
- **Isla:** si ya pasó 3 veces (`ISLA`) por esa celda, toma la salida que ha usado menos. Si hay empate, sigue la prioridad normal.

Mapa por celdas. Cada celda guarda 4 votos (N, E, S, O) en un diccionario de coordenadas.

- Cada lectura vota pared (+1) o libre (−1), con tope ±5 (`VOTO_MAX`). Así una lectura mala no borra lo que vieron las demás.
- Por donde el robot pasó queda libre con certeza.
- La celda actual sale de los mm que de verdad avanzó (no de lo que pidió).
- Cada color leído se anota en su celda; el amarillo marca la meta.

Persistencia. Al terminar la corrida se guarda `/app/datos/mapa.json` (paredes, colores y meta), con escritura atómica. Al leerlo tolera un archivo roto o vacío. Un toque largo del botón (más de 2 s) borra mapa y conteo.

Ruta corta. Con la meta en el mapa, la siguiente corrida calcula por BFS el camino más corto sobre pasos conocidos como libres (OLED `RAPIDO`). Si al avanzar no queda en la celda esperada, abandona la ruta y sigue explorando desde ahí.

Corrida. El botón corto inicia una corrida nueva (el robot debe estar en el inicio, siempre en la misma celda y mirando al mismo lado); otro toque la detiene.

Página web. Puerto 7000 por WiFi. Durante una corrida **no llama al Bridge**: muestra la última lectura que tomó el robot y su última foto, para no quitarle tiempo ni la cámara. Fuera de corrida lee directamente los sensores. Incluye el panel de calibración de colores, la vista de la cámara y un dibujo en vivo de los ToF con lo que haría el robot.

8. Hardware

En simple: una batería de 7.4 V alimenta los motores y la placa. Todos los sensores comparten dos cables de datos (I²C). El chasis lo diseñamos nosotros en 3D, con medidas exactas, y lo imprimimos. Comprobamos por computadora que ninguna pieza choque con otra antes de imprimir.

Subsistema	Implementación
Alimentación	Batería 2S (6.0–8.4 V) → interruptor → L298N (V_S) y VIN del UNO Q (7–24 V, reguladores conmutados)
Lógica de potencia	Regulador 78M05 del L298N (necesita ≳ 7 V). Entradas compatibles con 3.3 V (V_IH ≥ 2.3 V)
Bus I²C	400 kHz, alimentado a 3.3 V para no exponer los pines del Q a *pull-ups* de 5 V. Lleva 3 × VL53L0X, TCS34725, una OLED SSD1306 y, opcional, un MPU-6050 (0x68)
Pantalla	OLED 128 × 64 con driver propio. Muestra estado, conteos y un contador de *heartbeat* de la comunicación entre los procesadores
Chasis	OpenSCAD paramétrico de dos pisos, FDM (Bambu X2D), insertos de latón M3 y pedestal impreso con inserto M4. Interferencias verificadas por intersección CSG
Cambio de placa	Mismo mapa de pines en Q y R3 vía Proto Shield; solo cambia el puente de VCC (3V3 ↔ 5V)

Voltaje en los motores: el L298N es bipolar (Darlington) y cae ≈ 2–3 V. Con 7.4 V llegan ≈ 4.5–5 V a los motores. Es la principal ineficiencia del diseño; un puente MOSFET (TB6612FNG, DRV8833) la eliminaría.

9. Parámetros calibrables

Valores tal como están en `sketch.ino` y `main.py`.

Sketch (STM32)

Parámetro	Valor	Qué controla
`CELDA`	180 mm	Avance de una celda
`RANGO`	1000 mm	Con el frontal por debajo avanza por distancia; más lejos, por tiempo
`MS_CELDA`	600 ms	Tiempo por celda cuando no hay pared a la vista
`MS_90`	250 ms	Giro de 90° por tiempo (solo sin MPU-6050)
`VEL_EXPLORA` / `VEL_LENTO` / `VEL_GIRO` / `VEL_MIN`	140 / 100 / 130 / 80	PWM de 0 a 255
`PARED` / `PARED_F`	120 / 130 mm	Pared lateral / pared al frente
`D_FRENTE` / `D_CERCA`	130 / 10 mm	Alto al frente / retrocede si baja de 120 mm
`FRENA_K`	2 PWM/mm	Frenado proporcional al acercarse
`SALTO` / `TRAS_SALTO`	120 / 124 mm	Cambio brusco / avance posterior para centrarse en la celda abierta
`D_LADO_MIN`	90 mm	Solo corrige el centrado si un lateral lee menos
`D_LADO`	50 mm	Distancia lateral con el robot centrado
`KC` / `KD`	0.8 / 0.3	Centrado (posición / velocidad de acercamiento)
`KG`	2.0 PWM/°	Sostener rumbo en recta (con MPU-6050)
`ADELANTO`	6°	Corta el giro antes por la inercia (si se pasa, subir)
`AJUSTE_DER`	0	Corrige si un motor es más fuerte
`INV_IZQ` / `INV_DER`	true / true	Invierte el sentido del motor por software
`BRILLO_MIN`, `DIST_MAX`	1.3 / 0.02	Filtros del sensor de color

Python (Linux)

Parámetro	Valor	Qué controla
`FRENTE_LIBRE`	220 mm	El frente cuenta como salida
`VOTO_MAX`	5	Tope de votos por pared
`ISLA`	3 visitas	Pasa a la salida menos usada
`Y_COLOR`	36 mm	Del eje al sensor de color
`BLANCO_MIN`	170	Gris (0-255) a partir del cual un píxel cuenta como blanco
`ZONA_PASO`	(0.30, 0.55, 0.70, 1.00)	Piso por donde va a pasar, en fracción de la imagen
`BLANCO_MAX`	0.15	Fracción de blanco que activa el avance lento
Colores de referencia	de fábrica en `REFS`	Se miden en el panel web; se guardan en `/app/datos/colores.json`

10. Glosario

Término	Significado
MCU	Microcontrolador: computadora pequeña y puntual para controlar hardware
MPU	Microprocesador: computadora grande que corre un sistema operativo (Linux)
RPC	Llamada a procedimiento remoto: un procesador le pide a otro que ejecute una función
I²C	Bus de 2 cables (SDA datos, SCL reloj) que comparten varios sensores
ToF	*Time of Flight*: mide distancia por el tiempo que tarda la luz en ir y volver
PWM	Prender y apagar muy rápido para regular la potencia; 0 = apagado, 255 = máximo
Puente H	Circuito que deja girar un motor en ambos sentidos
Lazo abierto / cerrado	Sin / con medición del resultado para corregir
Control P	Corrige en proporción al error
Encoder	Sensor que cuenta las vueltas de la rueda
IMU / MPU-6050	Sensor de giro y aceleración; aquí se usa su giroscopio para medir el ángulo de los giros
BFS	Búsqueda en anchura: encuentra el camino más corto entre dos celdas
Rumbo	Hacia dónde apunta el robot, en grados
HSV	Forma de describir un color por tono, saturación y brillo
Bridge	Servicio que pasa las llamadas (RPC) entre Linux y el STM32
CSG	Operaciones booleanas entre sólidos 3D (unión, resta, intersección)

11. Evolución del diseño

La bitácora completa de cambios, problemas y soluciones está en `historial_cambios.md`. Los

cambios más importantes desde la V1:

- **V1:** movimiento por tiempo, sin encoders, con *flood fill* sobre un mapa de 16 × 16 celdas y respaldo en UNO R3 (ATmega328P).
- **Sensores:** 3 × VL53L0X reasignados de dirección I²C y sensor de color TCS34725 con calibración desde el panel web.
- **Cámara:** primero detectaba paredes para anticipar caminos; después buscó también el amarillo; con la cámara monocromática OV9281 solo detecta blanco (pared) en el piso por donde va a pasar.
- **Avance por distancia:** el frontal mide la celda, para a 130 mm y retrocede si queda muy cerca; sin pared va por tiempo.
- **Cambios bruscos** de los ToF (`SALTO`): el robot se centra en la celda abierta y la escanea.
- **Decisión:** pasó de *flood fill* con paredes de la cámara, y luego de reglas de pared, a una máquina de estados `SENSA → DECIDE → GIRA → AVANZA` con prioridad frente, izquierda, derecha.
- **Mapa:** votos ±5 por pared y colores por celda; ruta corta por BFS en la siguiente corrida.
- **Meta:** el STM32 para solo al confirmar el amarillo, sin esperar a Python.
- **Giros con MPU-6050** opcional: rumbo absoluto y rumbo sostenido en recta; sin él, por tiempo.
- **Web:** durante la corrida no usa el Bridge para no quitarle tiempo al robot.